

■ SecureApp

Documento Técnico – Fase 1: Blue Team

Proyecto 2: Construye, Ataca, Defiende

Nombre Completo	Correo Institucional	Rol
(Coordinador)		Coordinador
		Integrante
		Integrante
		Integrante

1. Introducción

El presente documento describe la arquitectura, decisiones de diseño y controles de seguridad implementados en **SecureApp**, una aplicación web desarrollada en el marco del Proyecto 2 de ISW-1013. El objetivo de la fase Blue Team es construir un sistema que resista ataques reales ejecutados por el equipo Red Team asignado, aplicando las mejores prácticas de la industria.

Objetivo: Construir una aplicación web segura con CRUD de productos y usuarios, control de acceso basado en roles (RBAC), API REST autenticada con JWT y un log de auditoría completo, cumpliendo todos los requerimientos funcionales y de seguridad del enunciado.

1.1 Tecnologías seleccionadas

Capa	Tecnología	Versión	Justificación de seguridad
Runtime	Node.js	20 LTS	Soporte activo hasta 2026, parches de seguridad frecuentes
Framework web	Express	4.19	Maduro, ampliamente auditado, middleware ecosystem rico
Base de datos	PostgreSQL	16	ACID, prepared statements nativos, sin riesgo SQLi con pg
Hashing	bcrypt	5.1	Algoritmo adaptativo con sal aleatoria integrada (RF-02)
Plantillas	EJS	3.1	Escapa HTML automáticamente con <%= %>, previene XSS
Sesiones	express-session	1.18	Cookies HttpOnly+Secure, timeout 5 min, regeneración post-login
JWT	jsonwebtoken	9.0	Firma HS256, expiración 1h, algoritmo none rechazado
Seguridad HTTP	Helmet	7.1	CSP, HSTS, X-Frame-Options DENY, nosniff, Referrer-Policy
CSRF	csrf	1.11	Token sincronizado en sesión para todos los formularios
Rate limiting	express-rate-limit	7.3	Bloqueo de IP tras 5 intentos fallidos por 5 minutos
Validación	express-validator	7.1	Validación en backend independiente del frontend
Contenedores	Docker Compose	3.9	Aislamiento, reproducibilidad, despliegue con 1 comando

2. Arquitectura de Seguridad

2.1 Diagrama del sistema

La aplicación sigue una arquitectura de tres capas con controles de seguridad en cada nivel:



2.2 Flujo de autenticación

Paso	Acción	Control de seguridad aplicado
1	Usuario envía credenciales	CSRF token validado, rate limit verificado
2	Servidor busca usuario en DB	Prepared statement paramétrico
3	Verifica contraseña	bcrypt.compare() — timing-safe, evita timing attacks
4	Regenera ID de sesión	session.regenerate() — previene Session Fixation
5	Guarda sesión encriptada	Cookie HttpOnly + Secure + SameSite=strict
6	Registra en audit_log	Evento LOGIN_EXITOSO/FALLIDO con IP y timestamp
7	Redirige al dashboard	Solo si todos los pasos anteriores son exitosos

3. Controles de Seguridad Implementados

3.1 RS-01 – Protección contra SQL Injection

Todas las consultas usan prepared statements del driver **pg**. El input del usuario NUNCA se concatena en strings SQL. Ejemplo:

```
db.query('SELECT * FROM usuarios WHERE username = $1', [username])
```

El ORM no se utiliza; se prefiere pg directo con parámetros posicionales para máxima transparencia y control.

3.2 RS-02 – Protección contra XSS

EJS escapa automáticamente todo output con `<%= %>`. La directiva `<%-` (HTML sin escapar) no se utiliza en ninguna vista. Adicionalmente, Helmet configura una política CSP estricta:

```
defaultSrc: ['self'] scriptSrc: ['self'] frameSrc: ['none'] objectSrc: ['none']
```

3.3 RS-03 – Protección CSRF

El middleware **csrf** genera un token único por sesión. Todos los formularios que realizan operaciones de escritura (crear, editar, eliminar, logout) incluyen `<input type="hidden" name="_csrf" value="...">`. La API REST está exenta porque usa JWT (stateless).

3.4 RS-04 – Gestión Segura de Sesiones

Cuatro controles combinados:

- **Timeout de inactividad:** maxAge=5min con rolling=true (se reinicia en cada request)
- **Prevención de Session Fixation:** req.session.regenerate() inmediatamente después del login exitoso
- **HttpOnly:** la cookie no es accesible desde JavaScript del cliente
- **Secure:** activo en producción (NODE_ENV=production), fuerza HTTPS
- **SameSite=strict:** capa adicional anti-CSRF

3.5 RS-05 – Seguridad en la API (JWT)

El endpoint POST /api/auth/login emite un JWT HS256 con expiración de 1 hora. Antes de verificar, el servidor decodifica el header del token y rechaza explícitamente el algoritmo "none". Se usa jsonwebtoken con opción **algorithms: ['HS256']** para denegar cualquier otro algoritmo.

3.6 RS-06 – Headers de Seguridad HTTP

Header	Valor configurado	Protección
Content-Security-Policy	default-src 'self'; frame-src 'none'	XSS, clickjacking, data injection

X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	MIME sniffing
Strict-Transport-Security	max-age=31536000; includeSubDomains	Downgrade HTTPS
Referrer-Policy	strict-origin-when-cross-origin	Fuga de información

3.7 RS-07 – Rate Limiting en Login

express-rate-limit bloquea la IP durante 5 minutos al detectar 5 intentos fallidos consecutivos. Aplica tanto al login web como al endpoint API /api/auth/login. Cada bloqueo se registra en el log de auditoría con el evento RATE_LIMIT_LOGIN e IP de origen.

Anexo A – Tabla de Resultados Blue Team

Código	Requerimiento	Estado	Evidencia / Observaciones
RF-01	App web con BD relacional	■ Exitoso	Express + PostgreSQL 16. Stack justificado en sección 1.1
RF-02	Hash seguro de contraseñas	■ Exitoso	bcrypt cost=12 en src/routes/users.js y src/routes/api/users.js
RF-03	CRUD Productos front+back	■ Exitoso	express-validator en backend + validación JS en frontend
RF-04	Usuarios + historial login+IP	■ Exitoso	Columnas ultimo_login y ultimo_login_ip actualizadas en cada login
RF-05	RBAC validado en backend	■ Exitoso	requireRole() middleware en cada ruta; acceso directo por URL retorna 403
RF-06	Log de auditoría completo	■ Exitoso	Tabla audit_log con todos los eventos solicitados + IP + timestamp
RF-07	API REST documentada con JWT	■ Exitoso	Endpoints /api/* documentados en api_docs/openapi.yaml (OpenAPI 3.0)
RS-01	Protección SQL Injection	■ Exitoso	100% prepared statements; prohibida concatenación SQL en el proyecto
RS-02	Protección XSS + CSP	■ Exitoso	EJS escapa con <%= %>; Helmet configura CSP strict en todas las respuestas
RS-03	Protección CSRF	■ Exitoso	csrf token en todos los forms de escritura; API exenta (JWT stateless)
RS-04	Sesión segura	■ Exitoso	rolling=true, maxAge=5min, regenerate() post-login, HttpOnly, Secure
RS-05	JWT correcto	■ Exitoso	HS256, exp=1h, algoritmo none rechazado, verificación en cada request
RS-06	Headers HTTP	■ Exitoso	CSP, X-Frame-Options DENY, nosniff, HSTS, Referrer-Policy via Helmet
RS-07	Rate limiting en login	■ Exitoso	5 intentos → bloqueo 5 min. Aplica en web y API. Registrado en audit_log

Anexo B – Casos de Prueba (mínimo 20)

Los siguientes 25 casos de prueba cubren los requerimientos funcionales y de seguridad:

CP-01	Módulo: Autenticación	Login exitoso con credenciales válidas
Precondición	Usuario admin existe	
Pasos	1. Ir a /auth/login 2. Ingresar admin/Admin1234! 3. Click Iniciar Sesión	
Resultado esperado	Redirige a /dashboard. Sesión iniciada. Evento LOGIN_EXITOSO en audit_log.	
CP-02	Módulo: Autenticación	Login fallido con contraseña incorrecta
Precondición	Usuario admin existe	
Pasos	1. Ir a /auth/login 2. Ingresar admin/wrongpass 3. Click Iniciar Sesión	
Resultado esperado	Permanece en login con mensaje "Credenciales incorrectas". Evento LOGIN_FALLIDO en audit_log.	
CP-03	Módulo: RS-07 Rate Limit	Bloqueo tras 5 intentos fallidos
Precondición	Ninguna	
Pasos	1. Enviar 5 POSTs a /auth/login con credenciales incorrectas	
Resultado esperado	Al 6to intento: HTTP 429 y mensaje de bloqueo 5 minutos. Evento RATE_LIMIT_LOGIN en audit_log.	
CP-04	Módulo: RS-04 Session Fixation	Regeneración del ID de sesión post-login
Precondición	Ninguna	
Pasos	1. Abrir DevTools → Application → Cookies 2. Anotar connect.sid antes del login 3. Hacer login exitoso 4. Verificar connect.sid	
Resultado esperado	El valor de connect.sid es diferente después del login.	
CP-05	Módulo: RS-04 Timeout	Expiración de sesión por inactividad
Precondición	Sesión activa	

Pasos	1. Hacer login 2. Esperar 5 minutos sin actividad 3. Intentar navegar	
Resultado esperado	Redirige a /auth/login. Sesión invalidada.	
CP-06	Módulo: RF-02 Bcrypt	Contraseñas almacenadas con hash bcrypt
Precondición	Acceso a BD	
Pasos	1. Crear usuario con contraseña Test123! 2. Consultar tabla usuarios en PostgreSQL	
Resultado esperado	La columna password_hash contiene \$2b\$12\$... (nunca texto plano).	
CP-07	Módulo: RF-03 Producto	Crear producto con datos válidos
Precondición	Sesión Registrador	
Pasos	1. Ir a /productos/nuevo 2. Llenar código PROD-001, nombre, cantidad=10, precio=500 3. Submit	
Resultado esperado	Producto creado. Redirige a /productos con mensaje de éxito. Evento PRODUCTO_CREADO en audit_log.	
CP-08	Módulo: RF-03 Producto	Crear producto con código duplicado
Precondición	Producto PROD-001 existente	
Pasos	1. Intentar crear otro producto con código PROD-001	
Resultado esperado	Error: "El código de producto ya existe." Producto no se crea.	
CP-09	Módulo: RF-03 Validación	Rechazo de cantidad negativa en producto
Precondición	Sesión Registrador	
Pasos	1. Intentar crear producto con cantidad=-1	
Resultado esperado	Error de validación en frontend y backend. No se crea el producto.	
CP-10	Módulo: RF-05 RBAC	Auditor no puede crear productos
Precondición	Sesión Auditor	
Pasos	1. Navegar a /productos/nuevo directamente por URL	
Resultado esperado	HTTP 403. Mensaje "No tienes permisos". Evento ACCESO_DENEGADO en audit_log.	

CP-11	Módulo: RF-05 RBAC	Registrador no puede eliminar usuarios
Precondición	Sesión Registrador	
Pasos	1. Hacer POST a /usuarios/1/eliminar	
Resultado esperado	HTTP 403. Operación denegada. Evento ACCESO_DENEGADO en audit_log.	
CP-12	Módulo: RF-05 RBAC	Auditor no puede acceder a auditoría
Precondición	Sesión Auditor	
Pasos	1. Navegar a /auditoria directamente por URL	
Resultado esperado	HTTP 403. Solo SuperAdmin puede ver el log de auditoría.	
CP-13	Módulo: RF-06 Auditoría	Registro de creación de usuario
Precondición	Sesión SuperAdmin	
Pasos	1. Crear un nuevo usuario desde /usuarios/nuevo	
Resultado esperado	Evento USUARIO_CREADO aparece en /auditoria con IP, username y timestamp.	
CP-14	Módulo: RS-01 SQL Injection	Intento de SQLi en login
Precondición	Ninguna	
Pasos	1. Ingresar username: ' OR '1'='1' -- 2. Cualquier contraseña	
Resultado esperado	Login falla. Sin error de BD. Prepared statement neutraliza el payload.	
CP-15	Módulo: RS-01 SQL Injection	Intento de SQLi en búsqueda de producto
Precondición	Sesión activa	
Pasos	1. Modificar URL: /productos?id=1 UNION SELECT...	
Resultado esperado	Error 404 o resultado vacío. Sin datos filtrados.	
CP-16	Módulo: RS-02 XSS Almacenado	Script en descripción de producto
Precondición	Sesión Registrador	
Pasos	1. Crear producto con descripción: alert(1)	
Resultado esperado	El script se muestra como texto literal. EJS lo escapa a <script>.	

CP-17	Módulo: RS-02 XSS Reflejado	Script en parámetro de URL
Precondición	Ninguna	
Pasos	1. Navegar a /?q=alert(1)	
Resultado esperado	El parámetro no se refleja sin escapar. Sin ejecución de script.	
CP-18	Módulo: RS-03 CSRF	Formulario externo sin token CSRF
Precondición	Sesión activa	
Pasos	1. Crear HTML externo con form que POSTea a /productos/1/eliminar 2. Abrir el HTML	
Resultado esperado	HTTP 403 CSRF token invalid. Operación rechazada.	
CP-19	Módulo: RS-05 JWT Alg None	Token JWT con algoritmo none
Precondición	Ninguna	
Pasos	1. Obtener JWT válido 2. Modificar header a alg:none y reenviar a /api/productos	
Resultado esperado	HTTP 401 "Algoritmo JWT no permitido."	
CP-20	Módulo: RS-05 JWT Expirado	Token JWT expirado
Precondición	Token de 1 hora	
Pasos	1. Esperar expiración del token (o manipular exp) 2. Enviar a /api/productos	
Resultado esperado	HTTP 401 "Token expirado."	
CP-21	Módulo: RS-06 Headers	Headers de seguridad presentes
Precondición	Servidor corriendo	
Pasos	1. Ejecutar: curl -I http://localhost:3000/	
Resultado esperado	Respuesta incluye CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, HSTS.	
CP-22	Módulo: RF-07 API	Login vía API y uso de JWT
Precondición	Servidor corriendo	
Pasos	1. POST /api/auth/login con JSON credentials 2. Usar token en Authorization: Bearer 3. GET /api/productos	

Resultado esperado	CP-22a: responde con JSON {token, ...}. CP-22b: lista de productos en JSON.	
CP-23	Módulo: RF-04 Historial	Registro de último login e IP
Precondición	Sesión admin	
Pasos	1. Hacer login 2. Ver tabla usuarios como SuperAdmin	
Resultado esperado	Columnas ultimo_login y ultimo_login_ip actualizadas con la IP real y timestamp actual.	
CP-24	Módulo: RS-07 API Rate Limit	Bloqueo en endpoint API login
Precondición	Ninguna	
Pasos	1. Enviar 5 POSTs consecutivos a /api/auth/login con credenciales incorrectas	
Resultado esperado	HTTP 429 en el 6to intento. Bloqueo por 5 minutos.	
CP-25	Módulo: Logout	Invalidación de sesión al salir
Precondición	Sesión activa	
Pasos	1. Click en "Salir" 2. Intentar navegar a /dashboard	
Resultado esperado	Cookie eliminada. Redirige a /auth/login. Sesión destruida en BD.	

4. Conclusiones

Lecciones aprendidas:

El desarrollo de SecureApp nos permitió comprender que la seguridad no es una característica que se agrega al final, sino una propiedad que debe diseñarse desde la arquitectura. Cada decisión técnica —desde el motor de plantillas hasta el manejo de sesiones— tiene implicaciones directas en la superficie de ataque.

Lo más difícil de implementar:

El control de acceso basado en roles (RBAC) resultó ser el rubro más complejo, no por su lógica de negocio, sino por garantizar que la validación ocurra en el backend en absolutamente cada ruta y endpoint de la API. Es tentador confiar solo en la UI para ocultar botones, pero el Red Team puede ignorar la interfaz completamente y atacar directamente las rutas HTTP. La solución fue crear middleware de RBAC reutilizable (`requireRole`, `requireRoleAPI`) y aplicarlo en cada definición de ruta.

La gestión segura de sesiones también representó un desafío conceptual: entender la diferencia entre Session Fixation (regenerar el ID post-login) y Session Hijacking (cookies `HttpOnly+Secure`) y implementar ambas protecciones correctamente.

Recomendación principal al Red Team:

Priorizar el análisis del RBAC mediante acceso directo a rutas restringidas por URL y API, ya que es el control de mayor impacto si falla. Usar Burp Suite para interceptar y modificar requests autenticados cambiando roles en el JWT.

Este documento fue elaborado con apoyo de Claude (Anthropic) para scaffold de código y documentación. Todo el contenido fue revisado y comprendido por el equipo.